Bits y E/S mapeada en memoria

Para la CPU, encender un LED es **escribir un bit en una dirección de memoria**; para el pin, ese bit son **5 V** (AVR) o **3,3 V** (ESP32).

Hardware · Software · código · Regla · decisión · Error · ojo · ◇ decisión

1 Binario → hex → decimal

ejemplo: el byte 1100 1000

1 Un byte: 8 bits con peso

Se numeran de **derecha a izquierda**: bit 0 a la derecha. El valor es la **suma de los pesos** de los bits en 1.

bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
1	**1**	0	0	**1**	0	0	0
128	64	32	16	8	4	2	1

2 Decimal: sumo los pesos de los 1

```
128 + 64 + 8 = 200
```

3 Hex: agrupo de 4 en 4

1 dígito hex = 4 bits. En C, `0x` es hexadecimal y `0b` es binario.

```
1100 | 1000 → C | 8 → 0xC8
```

4 Vuelta a binario: resto pesos de mayor a menor

```
200 − 128 − 64 − 8 = 0 → 1100 1000
```

UN DÍGITO HEX = 4 BITS

0000 **0**	0001 **1**	0010 **2**	0011 **3**	0100 **4**	0101 **5**	0110 **6**	0111 **7**
1000 **8**	1001 **9**	1010 **A**	1011 **B**	1100 **C**	1101 **D**	1110 **E**	1111 **F**

COMPUERTAS Y OPERADORES EN C

A	B	AND &	OR |	XOR ^	NOT A ~
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

0XCA & 0X0F: COLUMNA POR COLUMNA

```
  0xCA = 1100 1010
& 0x0F = 0000 1111
  ---------------
  0x0A = 0000 1010
```

CÓMO LEERLAS

AND: 1 solo si ambos son 1 · **OR**: 1 si alguno es 1 · **XOR**: 1 si son distintos · **NOT**: invierte. La CPU hace las 8 columnas en una instrucción.

2 ¿Qué hago con el bit n?

una máscara toca solo los bits que quiero

? **¿Qué quiero hacer con el bit n?**
Máscara = byte con 1 solo en los bits a tocar. `1 << 5` = `0010 0000` = `0x20`: dice «el bit 5».

Encender
OR con 1 da 1; OR con 0 deja igual.

```
REG |= (1 << n);
```

Apagar
`~` invierte la máscara; AND con 0 da 0.

```
REG &= ~(1 << n);
```

Invertir
XOR con 1 invierte; XOR con 0 deja igual.

```
REG ^= (1 << n);
```

Preguntar
Distinto de 0 solo si el bit está en 1.

```
if (REG & (1 << n))
```

ERROR CLÁSICO
`REG = (1 << 2);` usa `=` en vez de `|=`: **borra todos los demás bits**. Lo correcto: `REG |= (1 << 2);`

REGLAS DE LA MÁSCARA
Donde la máscara tiene 0, **el bit no cambia**. Encender los bits 0 y 1: `REG |= (1 << 0) | (1 << 1);` (máscara `0x03`).

3 DDR → PORT → estado del pin

el bit n de cada registro es el pin n del puerto

1 **DDRx: ¿entrada o salida?** MIRA EL BIT N
Registro de **dirección**. Bit en 1 = **salida**.

2 **PORTx: ¿qué valor?** MIRA EL BIT N
Con el pin como salida, el valor que sale. Como entrada, **activa la pull-up**.

Flotante
Entrada que lee **al azar**. Ej.: nada conectado.

```
DDR 0 · PORT 0
```

Pull-up
Entrada: suelto = 1, pulsado = 0. Ej.: botón a GND.

```
DDR 0 · PORT 1
```

Salida 0 V
Ej.: LED apagado.

```
DDR 1 · PORT 0
```

Salida 5 V
Ej.: LED encendido.

```
DDR 1 · PORT 1
```

PINX: LEER EL PIN
Registro que lee el **voltaje real** del pin: bit en 1 = el pin está en alto. (AVR: puertos B, C y D del ATmega328P.)

CON PULL-UP LA LÓGICA QUEDA INVERTIDA
Botón **suelto lee 1, pulsado lee 0**: el botón une el pin con GND.

CONVERSIONES DE CLASE (RESUELTAS)

DATO	BINARIO	DECIMAL	HEX
`1000 0001`	1000 0001	129	0x81
`0xFF`	1111 1111	255	0xFF
`0x08`	0000 1000 → **bit 3** en 1	8	0x08
`200`	1100 1000	200	0xC8
`0x20 | 0x01`	0010 0001 → bits 5 y 0	33	0x21

PREDICE EL PIN (RESUELTO)

CASO	CONFIGURACIÓN	RESULTADO
A	DDRB5 = 1, PORTB5 = 1	Salida en alto: **5 V**
B	DDRB5 = 1, PORTB5 = 0	Salida en bajo: **0 V**
C	DDRD2 = 0, PORTD2 = 1, botón pulsado a GND	Pull-up, pulsado: **PIND2 = 0**
D	DDRD2 = 0, PORTD2 = 0, nada conectado	Flotante: **valor al azar**

ARDUINO VS. REGISTROS (AVR)

FUNCIONES DE ARDUINO	REGISTROS (LO QUE PASA POR DENTRO)
`pinMode(13, OUTPUT);`	`DDRB |= (1 << 5);`
`digitalWrite(13, HIGH);`	`PORTB |= (1 << 5);`
`digitalWrite(13, LOW);`	`PORTB &= ~(1 << 5);`
`pinMode(2, INPUT_PULLUP);`	`DDRD &= ~(1 << 2); PORTD |= (1 << 2);`
`int b = digitalRead(2);`	`int b = (PIND & (1 << 2)) != 0;`

Pin 13 del Uno = PB5; pin 2 = PD2. Las funciones son más **legibles y portables**; los registros, más **rápidos** y muestran qué ocurre en el chip.

RESPUESTAS · TICKET DE SALIDA

01 *¿Cuánto es 0x81 en binario?*
`0x81` = `8` `1` = `1000 0001` (decimal 129).

02 *Escribe la línea que apaga el bit 4 de PORTB.*
`PORTB &= ~(1 << 4);`

03 *Con DDRD2 = 0 y PORTD2 = 1, ¿qué lee PIND2 con el botón pulsado a GND?*
0. Es entrada con pull-up; el botón une el pin con GND, así que pulsado lee bajo.